SOFTWARE QUALITY ASSURANCE · CHAPTER 6

Development and Quality Plans

ROADMAP

Chapter Roadmap — 8 Parts

1. Introduction — Why Development & Quality Plans Matter
2. Development Plan & Quality Plan Objectives
3. Elements of the Development Plan
4. Elements of the Quality Plan
5. Building the Development Plan — Process & Practice
6. Building the Quality Plan — Process & Practice
7. Development & Quality Plans for Small and Internal Projects
8. Integration — How the Two Plans Work Together

PART 1 OF 8

Introduction — Why Development & Quality Plans Matter

Two documents that turn a proposal's promises into an executable, verifiable plan.

Two Documents, One Project

- Once a contract is signed, its high-level commitments still need to become a concrete, day-to-day plan for building and verifying the software.
- The development plan answers one question: what will we build, in what order, with which people and tools?
- The quality plan answers a different question: how will we know, at each step, that what we're building is actually good?
- Treating these as two distinct documents — rather than folding quality into a single project plan — keeps quality visible instead of buried as an afterthought.

What Happens Without a Plan

- Without a documented development plan, schedule and task assignment default to whoever speaks loudest in the daily stand-up — consistent, but not necessarily correct.
- Without a documented quality plan, testing and review happen only when someone remembers, rather than at defined checkpoints.
- Both gaps are invisible in a project's early weeks and become painfully visible only as integration, testing, or delivery approaches.
- Teams without written plans often discover, too late, that everyone had a slightly different idea of what 'done' meant.

Why Development and Quality Plans Matter

Software Development Plans and Software Quality Assurance Plans are essential because they provide a clear direction for both project execution and quality management. Without these plans, software projects are often managed based on assumptions rather than documented strategies, resulting in inconsistent development practices and increased project uncertainty.

Development and quality plans help organizations achieve the following objectives:

- **Provide clear project direction:** Establish well-defined project objectives, deliverables, milestones, and responsibilities so that every stakeholder understands the project's purpose and expected outcomes.
- **Improve communication among stakeholders:** Define communication procedures, reporting mechanisms, and information-sharing practices that facilitate collaboration between customers, developers, testers, project managers, and senior management.

•**Support effective resource management:** Identify the personnel, hardware, software tools, financial resources, and infrastructure required to complete the project efficiently.

•**Reduce project risks:** Identify potential technical, managerial, financial, operational, and business risks early in the project and define appropriate mitigation strategies before development begins.

•**Ensure consistent software quality:** Establish quality standards, review procedures, testing strategies, and quality metrics that are applied consistently throughout the Software Development Life Cycle.

•**Improve project monitoring and control:** Enable project managers to measure progress against predefined schedules, budgets, quality objectives, and performance indicators.

•**Facilitate compliance with standards:** Ensure that software development activities comply with organizational policies, contractual obligations, and international software engineering standards such as ISO and IEEE.

•**Increase customer satisfaction:** Deliver software products that satisfy customer requirements, perform reliably, and meet agreed quality expectations.

What is a Software Development Plan (SDP)?

A **Software Development Plan (SDP)** is a formal management document that describes how a software project will be organized, executed, monitored, controlled, and completed throughout its lifecycle. It serves as the primary roadmap for project execution by defining project objectives, schedules, resources, responsibilities, budgets, and management procedures.

The Software Development Plan enables project managers to coordinate project activities effectively while ensuring that all stakeholders share a common understanding of project expectations.

Major Components of a Software Development Plan

•**Project Objectives:** Define the business goals, expected outcomes, and success criteria of the project. These objectives provide a clear direction for the project team and establish measurable targets for evaluating project success.

•**Project Scope:** Specify the boundaries of the project by identifying the software features, functions, deliverables, assumptions, constraints, and exclusions. A clearly defined scope helps prevent misunderstandings and uncontrolled scope expansion.

•**Development Methodology:** Describe the software development approach that will be used, such as Waterfall, Agile, Scrum, Spiral, DevOps, or Rapid Application Development (RAD). The selected methodology influences planning, scheduling, documentation, communication, and quality assurance activities.

•**Project Schedule:** Define the project timeline, including milestones, task dependencies, development phases, deadlines, and expected completion dates. A realistic schedule enables effective progress monitoring and timely delivery.

•**Resource Allocation:** Identify the human resources, development tools, hardware, software, financial resources, and infrastructure required to complete the project successfully.

•**Team Responsibilities:** Clearly assign roles, duties, responsibilities, and reporting relationships among project managers, software developers, business analysts, testers, quality assurance engineers, and other stakeholders.

•**Budget Estimates:** Provide detailed financial estimates covering personnel costs, software licenses, hardware acquisition, training, maintenance, contingency funds, and operational expenses.

•**Risk Management:** Identify potential project risks and define strategies for preventing, mitigating, monitoring, and responding to technical, managerial, financial, and operational risks.

•**Communication Strategy:** Establish communication channels, reporting procedures, meeting schedules, documentation standards, and decision-making processes to facilitate collaboration among stakeholders.

•**Deliverables and Milestones:** Identify the software products, documentation, reports, prototypes, training materials, and major milestones that will be produced throughout the project lifecycle.

What is a Software Quality Assurance Plan (SQAP)?

A **Software Quality Assurance Plan (SQAP)** is a formal quality management document that specifies the standards, procedures, reviews, testing activities, audits, quality metrics, and responsibilities required to ensure that software products satisfy customer requirements and organizational quality standards.

Unlike software testing, which primarily detects defects after implementation, the SQAP focuses on **preventing defects** by integrating quality assurance activities into every phase of software development.

Major Components of a Software Quality Assurance Plan

•**Quality Objectives:** Define the quality goals that the software product must achieve, including reliability, maintainability, usability, security, efficiency, portability, scalability, and customer satisfaction.

•**Applicable Standards:** Identify the organizational, national, and international standards that will guide software development and quality assurance activities, including ISO 9001, ISO/IEC 25010, IEEE standards, and organizational quality policies.

•**Review Activities:** Specify formal review activities such as requirements reviews, design reviews, code inspections, walkthroughs, peer reviews, and management reviews to identify defects before implementation.

•**Testing Strategy:** Describe the testing approach, including unit testing, integration testing, system testing, regression testing, performance testing, security testing, usability testing, and user acceptance testing.

•**Quality Metrics:** Define measurable indicators used to evaluate software quality and project performance, such as defect density, test coverage, code complexity, customer satisfaction, defect removal efficiency, and reliability.

•**Audit Schedule:** Establish the timing and frequency of internal and external quality audits to verify compliance with software engineering standards and organizational procedures.

•**Defect Reporting Procedures:** Define the process for identifying, recording, classifying, assigning, tracking, correcting, verifying, and closing software defects throughout the project.

•**Acceptance Criteria:** Specify measurable conditions that must be satisfied before the customer formally accepts the software product, ensuring compliance with contractual and quality requirements.

•**Roles and Responsibilities:** Clearly define the quality-related responsibilities of project managers, software developers, testers, quality assurance engineers, reviewers, auditors, customers, and senior management.

Relationship Between Development Planning and Quality Planning

Although the Software Development Plan and Software Quality Assurance Plan focus on different aspects of project management, they work together to achieve a common objective: the successful delivery of high-quality software.

Software Development Plan (SDP)	Software Quality Assurance Plan (SQAP)
Defines how the project will be planned, organized, executed, monitored, and controlled throughout the Software Development Life Cycle.	Defines how software quality will be achieved, measured, monitored, and continuously improved throughout the project.
Focuses on project management activities such as scheduling, budgeting, resource allocation, communication, and risk management.	Focuses on quality management activities including reviews, inspections, audits, testing, quality standards, and defect prevention.
Coordinates technical activities and project execution.	Coordinates quality assurance and quality control activities.
Measures project success based on scope, schedule, budget, and deliverables.	Measures software quality using standards, metrics, testing results, defect analysis, and customer satisfaction.

Benefits of Development and Quality Planning

Organizations that prepare comprehensive development and quality plans experience numerous technical, managerial, financial, and organizational benefits.

- **Improved Project Planning:** Enables systematic organization of project activities, schedules, budgets, and resources, leading to more predictable project outcomes.
- **Better Resource Utilization:** Ensures that personnel, equipment, development tools, and financial resources are allocated efficiently throughout the project.
- **Reduced Project Risks:** Identifies potential project risks early and establishes mitigation strategies before they affect project performance.

- **Improved Communication:** Establishes standardized communication procedures that improve collaboration among project stakeholders.
- **Higher Software Quality:** Integrates quality assurance activities throughout the development process, reducing software defects and improving product reliability.
- **Enhanced Project Monitoring:** Provides measurable milestones, quality metrics, and performance indicators that enable continuous monitoring and project control.
- **Regulatory and Standards Compliance:** Supports compliance with organizational policies, contractual obligations, and internationally recognized software engineering standards.
- **Increased Customer Satisfaction:** Improves the likelihood of delivering software that meets customer expectations regarding functionality, quality, performance, security, and reliability.

Consequences of Poor Planning

Organizations that fail to prepare effective Software Development Plans and Software Quality Assurance Plans often experience significant project difficulties.

- **Unclear Project Objectives:** Team members may not fully understand the project's purpose, resulting in inconsistent development efforts.
- **Unrealistic Schedules:** Poor estimation often leads to missed deadlines, delayed delivery, and increased development costs.
- **Budget Overruns:** Inadequate financial planning can result in unexpected expenses and insufficient funding.
- **Scope Creep:** Poorly defined project boundaries frequently lead to uncontrolled additions of new requirements during development.

- **Resource Shortages:** Failure to allocate appropriate personnel, tools, and infrastructure reduces project productivity.
- **Poor Software Quality:** Insufficient quality planning increases software defects, testing problems, and maintenance costs.
- **Communication Failures:** Lack of communication planning causes misunderstandings among stakeholders and delays in decision-making.
- **Customer Dissatisfaction:** Projects that fail to meet customer expectations often result in complaints, contractual disputes, or project cancellation.

The Cost of Planning Gaps

What tends to happen when neither plan is written down.

Rework

Tasks redone because responsibilities or scope were never clearly assigned

Late Defects

Quality issues discovered at delivery instead of at a scheduled checkpoint

Drift

Schedules that silently slip because no baseline plan existed to slip against

PART 2 OF 8

Development Plan & Quality Plan Objectives

Objectives of a Software Development Plan (SDP)

The primary purpose of the Software Development Plan is to establish a structured approach for managing the software project from initiation to completion.

Major Objectives of the Software Development Plan

- **Define Project Objectives:** Clearly establish the business goals, expected outcomes, project deliverables, and success criteria so that every stakeholder understands what the project is intended to achieve.
- **Define Project Scope:** Specify the boundaries of the project by identifying included features, excluded functionality, assumptions, constraints, and expected deliverables to prevent misunderstandings and scope creep.
- **Develop a Realistic Project Schedule:** Prepare a practical project timeline that includes milestones, task dependencies, development phases, deadlines, and expected completion dates to support effective project monitoring.
- **Allocate Project Resources:** Identify and assign the human resources, hardware, software tools, infrastructure, financial resources, and facilities required to complete the project successfully. 2

- **Assign Roles and Responsibilities:** Clearly define the responsibilities of project managers, software developers, business analysts, testers, quality assurance engineers, customers, and other stakeholders to improve accountability and collaboration.
- **Estimate Project Costs:** Prepare detailed cost estimates covering personnel expenses, hardware, software licenses, training, maintenance, contingency funds, and operational costs to support financial planning.
- **Manage Project Risks:** Identify potential technical, financial, operational, organizational, and managerial risks and develop strategies to minimize their impact on project success.
- **Establish Communication Procedures:** Define reporting structures, communication channels, meeting schedules, documentation practices, and decision-making processes to facilitate effective collaboration.
- **Monitor Project Progress:** Establish measurable milestones, performance indicators, and reporting mechanisms that enable project managers to track progress and identify deviations from the project plan.
- **Support Successful Project Delivery:** Provide a structured framework that enables the project team to complete the software project on time, within budget, and according to agreed requirements.

Major Objectives of the Software Quality Assurance Plan

•**Establish Quality Objectives:** Define measurable quality goals for the software product, including reliability, security, usability, maintainability, efficiency, portability, scalability, and customer satisfaction.

•**Ensure Compliance with Standards:** Specify the quality standards, organizational procedures, industry regulations, and software engineering practices that must be followed throughout the project.

•**Prevent Software Defects:** Implement preventive quality assurance activities such as reviews, inspections, walkthroughs, audits, and process improvements to detect problems before implementation.

•**Define Testing Activities:** Establish a comprehensive testing strategy that includes unit testing, integration testing, system testing, regression testing, security testing, performance testing, usability testing, and user acceptance testing.

- **Establish Quality Metrics:** Define measurable indicators that evaluate software quality and process performance, such as defect density, test coverage, reliability, customer satisfaction, and defect removal efficiency.
- **Conduct Quality Reviews and Audits:** Schedule formal quality reviews, management reviews, process audits, and compliance assessments to verify that project activities conform to established standards.
- **Manage Software Defects:** Define procedures for recording, classifying, assigning, tracking, correcting, verifying, and closing software defects throughout the Software Development Life Cycle.
- **Ensure Customer Satisfaction:** Verify that the software product satisfies functional requirements, non-functional requirements, contractual obligations, and customer expectations before final delivery.
- **Promote Continuous Improvement:** Encourage continuous evaluation and improvement of software development processes through quality measurements, lessons learned, corrective actions, and process optimization.
- **Support High-Quality Software Delivery:** Ensure that the final software product meets agreed quality standards while minimizing defects, maintenance costs, and operational failures.

Comparison of Development Plan Objectives and Quality Plan Objectives

Software Development Plan Objectives	Software Quality Assurance Plan Objectives
Define project objectives and scope.	Define software quality objectives and standards.
Plan project schedule and milestones.	Plan reviews, inspections, audits, and testing activities.
Allocate project resources.	Ensure compliance with quality standards.
Estimate project costs.	Prevent software defects through quality assurance activities.
Assign project responsibilities.	Define quality-related roles and responsibilities.
Manage project risks.	Measure and monitor software quality using quality metrics.
Coordinate project communication.	Manage defects and continuous quality improvement.
Deliver the project successfully.	Deliver high-quality software products.

Benefits of Clearly Defined Objectives

Organizations that establish clear Development Plan and Quality Plan objectives experience numerous advantages.

- **Improved Project Direction:** Clearly defined objectives provide a common understanding of project goals, enabling all stakeholders to work toward the same outcomes.
- **Better Decision-Making:** Well-established objectives help project managers evaluate alternatives, prioritize activities, and make informed project decisions.
- **Enhanced Communication:** Clear objectives improve communication among project teams, customers, management, and external stakeholders by reducing misunderstandings.
- **Reduced Project Risks:** Identifying objectives early enables organizations to recognize potential risks and implement preventive measures before development begins.

- **Higher Software Quality:** Quality objectives ensure that reviews, testing, audits, and process improvements are integrated into every stage of software development.
- **Improved Resource Utilization:** Clearly defined objectives support efficient allocation of personnel, equipment, software tools, budgets, and infrastructure.
- **Greater Customer Satisfaction:** Projects guided by well-defined objectives are more likely to deliver software products that satisfy customer requirements and quality expectations.
- **Successful Project Delivery:** Clear objectives improve project coordination, reduce uncertainty, and increase the probability of delivering software on time, within budget, and according to agreed specifications.

DEVELOPMENT OBJECTIVE 1

Define Scope & Deliverables

Establishing, precisely, what the project will produce — and just as importantly, what it will not.

WHY IT MATTERS

- An undefined boundary is where scope creep quietly enters.
- Deliverables need to be concrete enough that 'complete' has an unambiguous meaning.
- This objective is what everything else in the development plan is built on top of.

IN PRACTICE

Example: rather than 'a reporting module,' the plan lists the exact reports, their formats, and their data sources.

DEVELOPMENT OBJECTIVE 2

Establish Schedule & Milestones

Turning the deliverables into a realistic sequence of tasks with dates the team can be held to.

WHY IT MATTERS

- A schedule with no milestones offers no early warning when things start to slip.
- Milestones should be tied to verifiable outputs, not just elapsed time.
- Sequencing reveals dependencies that aren't obvious from a deliverables list alone.

IN PRACTICE

Example: 'authentication module complete' is a milestone; 'two weeks of authentication work' is not.

DEVELOPMENT OBJECTIVE 3

Allocate Resources & Responsibilities

Assigning named people, tools, and infrastructure to each part of the plan — not leaving it implicit.

WHY IT MATTERS

- Unassigned tasks are the ones most likely to be forgotten under pressure.
- Resource allocation surfaces conflicts, like the same person assigned to two milestones at once, while they're still fixable.
- Clarity here is what makes the schedule credible in the first place.

IN PRACTICE

Example: naming which engineer owns the payment integration, not just noting that 'the team' will handle it.

DEVELOPMENT OBJECTIVE 4

Manage Development Risk

Naming what could derail the schedule or deliverables, and planning a response before it happens.

WHY IT MATTERS

- Development risk is different from quality risk — it's about whether the work gets done, not whether it's correct.
- Risks named early can be scheduled around; risks discovered mid-sprint usually can't.
- A development plan with no risk section implicitly assumes nothing will go wrong — rarely a safe assumption.

IN PRACTICE

Example: flagging that a third-party API the schedule depends on is still in beta, and planning a fallback.

QUALITY OBJECTIVE 1

Define Measurable Quality Goals

Translating 'good software' into specific, checkable targets for this project.

WHY IT MATTERS

- Unmeasurable quality goals can't be verified — only felt, which isn't reliable at scale.
- Goals should reflect what actually matters to this project's users, not a generic template.
- Without this objective, the rest of the quality plan has nothing concrete to aim at.

IN PRACTICE

Example: 'response time under 200ms for 95% of requests' rather than 'the system should be fast.'

QUALITY OBJECTIVE 2

Define QA Activities & Checkpoints

Specifying which reviews, audits, and checks happen, and when, throughout the project.

WHY IT MATTERS

- Checkpoints turn quality from a single end-of-project gate into an ongoing, lower-risk process.
- Each checkpoint should map to a specific development milestone, not float independently.
- Without scheduled checkpoints, review only happens when someone remembers to ask for it.

IN PRACTICE

Example: scheduling a design review before coding on the payment module begins, not after it's finished.

QUALITY OBJECTIVE 3

Define Testing & Defect Management

Establishing how the software will be tested, and how found defects will be tracked and resolved.

WHY IT MATTERS

- A quality plan silent on defect handling leaves 'what happens when we find a bug' undefined until the first bug appears.
- Testing approach should be scoped to the project's actual risk profile, not applied uniformly everywhere.
- Defect severity and resolution timelines should be agreed before defects start arriving, not negotiated in the moment.

IN PRACTICE

Example: defining that critical defects block release, while minor cosmetic issues can ship with a tracked follow-up.

QUALITY OBJECTIVE 4

Define Standards & Configuration Management

Establishing the conventions, standards, and version-control discipline the whole team will follow.

WHY IT MATTERS

- Inconsistent coding or documentation standards make review slower and less reliable.
- Configuration management determines whether 'which version did we actually test' has a confident answer.
- This objective is what keeps the other three objectives auditable after the fact.

IN PRACTICE

Example: specifying the branching strategy and the coding standard the team will follow, referenced by name, not left implicit.

PART 3 OF 8

Elements of the Development Plan

Eight components that turn the plan's objectives into a document someone can actually follow.

What are the Elements of a Software Development Plan?

The **Elements of a Software Development Plan** are the major sections or components that collectively describe how a software project will be organized, managed, executed, monitored, and controlled throughout its lifecycle.

Each element provides specific information that supports project planning and serves as a reference for project managers, developers, quality assurance engineers, customers, and other stakeholders.

No.	Development Plan Element	Description	Purpose / Importance
1	**Project Overview**	Provides a general description of the software project, including its background, objectives, business justification, stakeholders, and expected benefits.	Establishes a common understanding of the project's purpose and expected outcomes among all stakeholders.
2	**Project Scope**	Defines the boundaries of the project by specifying the features, functions, deliverables, assumptions, constraints, inclusions, and exclusions.	Prevents scope creep, reduces misunderstandings, and ensures all stakeholders have a shared understanding of what will and will not be delivered.
3	**Project Organization**	Describes the organizational structure of the project, including project roles, responsibilities, reporting relationships, and authority levels.	Improves accountability, coordination, communication, and decision-making throughout the project lifecycle.
4	**Development Methodology**	Specifies the software development approach to be used, such as Waterfall, Agile, Scrum, Spiral, DevOps, or RAD.	Provides a structured framework for organizing, executing, and controlling software development activities.
5	**Work Breakdown Structure (WBS)**	Divides the project into smaller, manageable tasks and work packages that can be assigned, monitored, and controlled.	Simplifies project planning, scheduling, resource allocation, and progress tracking.

6	**Project Schedule**	**Defines the project timeline, milestones, task dependencies, critical path, deadlines, and expected completion dates.**	**Ensures timely project completion and enables effective monitoring of project progress.**
7	**Resource Plan**	Identifies the human resources, hardware, software, infrastructure, development tools, and financial resources required for the project.	Ensures adequate resources are available throughout the project to support successful execution.
8	**Budget Plan**	Estimates all project costs, including personnel, hardware, software, training, maintenance, contingency funds, and operational expenses.	Supports financial planning, budget control, and cost management while minimizing financial risks.
9	**Risk Management Plan**	Identifies potential technical, managerial, operational, financial, security, and business risks together with mitigation strategies.	Reduces project uncertainty and prepares the team to respond effectively to potential problems before they affect the project.
10	**Communication Plan**	Defines communication channels, reporting procedures, meeting schedules, documentation standards, stakeholder communication, and escalation procedures.	Promotes effective communication, collaboration, transparency, and timely decision-making among project stakeholders.

11	**Configuration Management Plan**	**Defines procedures for managing software versions, project documentation, source code, baselines, and configuration items.**	**Maintains software integrity, supports version control, and ensures consistency throughout development.**
12	Quality Management Plan	Specifies quality objectives, standards, reviews, inspections, testing activities, quality metrics, and quality assurance procedures.	Ensures software products comply with customer requirements and organizational quality standards.
13	Procurement Plan	Describes the acquisition of external products, software licenses, hardware, consulting services, and vendor management activities.	Ensures timely procurement of required resources and effective management of suppliers and contracts.
14	Change Management Plan	Defines procedures for requesting, evaluating, approving, implementing, and monitoring project changes.	Controls scope changes, minimizes project disruption, and ensures that all changes are properly evaluated before implementation.
15	Project Monitoring and Reporting	Defines performance monitoring methods, project status reporting, progress tracking, issue management, and performance measurement.	Enables continuous monitoring of project performance, early detection of problems, and informed management decisions.

PART 4 OF 8

Elements of the Quality Plan

What are the Elements of a Software Quality Assurance Plan?

The **Elements of a Software Quality Assurance Plan (SQAP)** are the major components that define how software quality will be planned, managed, monitored, measured, and continuously improved throughout the project.

Each element addresses a specific aspect of quality management and collectively provides a structured framework for implementing Software Quality Assurance activities.

No.	Quality Plan Element	Description	Purpose / Importance
1	**Quality Objectives**	Defines the quality goals that the software product and development process must achieve, including reliability, maintainability, usability, security, efficiency, portability, scalability, and customer satisfaction.	Establishes measurable quality targets that guide all quality assurance activities and ensure the final software satisfies customer expectations.
2	**Applicable Standards and Procedures**	Identifies the organizational policies, coding standards, quality procedures, regulatory requirements, and international standards (e.g., ISO 9001, ISO/IEC 25010, IEEE standards) that govern the project.	Ensures consistency, regulatory compliance, and adherence to recognized software engineering best practices.
3	**Software Quality Assurance Organization**	Defines the organizational structure of the quality assurance function, including QA managers, quality engineers, auditors, reviewers, and testers together with their responsibilities.	Establishes accountability, authority, and effective coordination of all quality assurance activities.
4	**Review and Inspection Plan**	Specifies formal technical reviews, peer reviews, walkthroughs, code inspections, design inspections, requirements reviews, and management reviews that will be conducted throughout development.	Detects defects early in the SDLC, reducing the cost and effort required to correct them later.
5	**Testing Strategy and Test Plan**	Defines the overall testing approach, including unit testing, integration testing, system testing, regression testing, performance testing, security testing, usability testing, and user acceptance testing.	Ensures that software functionality, performance, reliability, and security are thoroughly verified before deployment.

6	**Configuration Management**	**Defines procedures for identifying, controlling, versioning, and maintaining software configuration items such as source code, documentation, test scripts, and project baselines.**	**Preserves software integrity, supports version control, and ensures consistency across development environments.**
7	**Defect Management Plan**	Describes how software defects will be identified, classified, recorded, assigned, tracked, corrected, verified, and closed throughout the project lifecycle.	Provides a systematic process for managing defects and continuously improving software quality.
8	**Quality Metrics and Measurement**	Specifies measurable indicators such as defect density, test coverage, code coverage, defect removal efficiency, customer satisfaction, reliability, and Mean Time Between Failures (MTBF).	Enables objective evaluation of software quality and supports evidence-based quality improvement.
9	**Quality Audits**	Defines the schedule, scope, procedures, responsibilities, and reporting mechanisms for conducting internal and external quality audits.	Verifies compliance with quality standards, organizational procedures, contractual obligations, and regulatory requirements.
10	**Risk Management for Quality**	Identifies quality-related risks such as inadequate testing, poor documentation, process non-compliance, security vulnerabilities, and technology limitations, together with mitigation strategies.	Minimizes quality failures and enables proactive management of risks affecting software quality.

11	**Training Plan**	**Identifies quality-related training requirements for project personnel, including software standards, testing techniques, development tools, quality procedures, and process improvement practices.**	**Ensures that all team members possess the knowledge and skills required to implement quality assurance effectively.**
12	Supplier Quality Management	Defines procedures for evaluating, selecting, monitoring, and controlling the quality of software vendors, subcontractors, consultants, and third-party software providers.	Ensures that externally acquired products and services satisfy organizational quality requirements.
13	Documentation Control	Establishes procedures for preparing, reviewing, approving, distributing, updating, storing, and maintaining software documentation and quality records.	Ensures that project documents remain accurate, current, controlled, and accessible throughout the project lifecycle.
14	Acceptance Criteria	Defines the measurable functional, performance, security, usability, reliability, and contractual conditions that must be satisfied before the software is accepted by the customer.	Provides objective criteria for determining whether the software product meets agreed quality requirements.
15	Continuous Improvement Plan	Defines procedures for collecting lessons learned, analyzing quality performance, implementing corrective actions, preventive actions, process optimization, and organizational learning.	Promotes continuous improvement of software processes, product quality, and organizational performance.

PART 5 OF 8

Building the Development Plan — Process & Practice

From a blank page to a plan the team can actually execute against.

The Development Planning Process

The process of building a Software Development Plan consists of several sequential activities.

1 Business Need Identified

- Identify the problem or opportunity.
- Understand business objectives and expected outcomes.
- Define high-level needs and align with strategic goals.

2 Project Initiation

- Define the project vision and objectives.
- Identify key stakeholders and roles.
- Establish initial project charter and scope boundaries.

3 Requirements Gathering

- Elicit and document functional and non-functional requirements.
- Engage stakeholders to understand needs.
- Validate and prioritize requirements.

4 Feasibility Study

- Analyze technical, economic and operational feasibility.
- Identify constraints and dependencies.
- Determine if the project is viable and worth pursuing.

5 Project Scope Definition

- Define the project scope and deliverables.
- Document what is included and excluded.
- Establish acceptance criteria and success metrics.

6 Resource Planning

- Identify required human resources and skills.
- Determine tools, technology and infrastructure needs.
- Plan for procurement and resource allocation.

7 Schedule Development

- Break down work into tasks and milestones.
- Estimate durations and dependencies.
- Create a realistic project timeline.

8 Budget Estimation

- Estimate costs for resources, tools and other needs.
- Identify cost assumptions and constraints.
- Develop the project budget baseline.

9 Risk Assessment

- Identify potential risks and uncertainties.
- Analyze probability and impact.
- Plan mitigation and contingency strategies.

10 Quality Planning

- Define quality objectives and standards.
- Plan quality assurance activities and metrics.
- Establish review and testing strategies.

11 Review & Approval

- Review the plan with stakeholders.
- Validate scope, budget, schedule and risks.
- Obtain formal approval to proceed.

12 Software Development Begins

- Kick off the project.
- Execute the plan and begin development activities.
- Monitor progress and adapt as needed.

Outcome:

A well-structured development plan ensures clarity, reduces risks, optimizes resources, and sets a strong foundation for successful software development.

- Clear Objectives & Scope
- Optimal Resource Utilization
- Managed Risks & Quality
- On-time & On-budget Delivery
- Successful Project Outcomes

Development Plan: Common Mistakes vs. Good Practice

	Common Mistake	Good Practice
Scope	Deliverables described in vague terms	Deliverables listed with concrete acceptance criteria
Schedule	Dates set before dependencies are mapped	Dependencies mapped first, dates follow
Staffing	Tasks assigned to ‘the team’	Tasks assigned to named individuals
Risk	Risk section written once, then ignored	Risk list reviewed at every milestone

PART 6 OF 8

Building the Quality Plan — Process & Practice

From quality goals to a plan reviewers and testers can actually follow.

Who Builds the Quality Plan

- Primary ownership typically sits with a QA lead or the quality assurance function, working closely with the project manager.
- Technical leads contribute input on where the highest-risk, most review-worthy parts of the system are.
- The quality plan should be drafted alongside the development plan, not after it — quality checkpoints need to be scheduled against real milestones.

Building the Quality Plan — Process & Practice

A systematic approach to define quality objectives, plan quality activities, and ensure continuous improvement throughout the project.

1 Understand the Project Context

- Review project objectives, scope, deliverables, and constraints.
- Understand customer needs, expectations, and success criteria.
- Identify applicable standards, policies, and regulations.

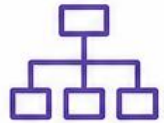
2 Define Quality Objectives

- Establish clear, measurable quality objectives.
- Align objectives with business goals and customer needs.
- Define quality success criteria and targets.

3 Identify Quality Requirements

- Identify product and process quality requirements.
- Determine quality attributes (reliability, performance, usability, security, etc.).
- Map requirements to standards and regulations.

4 Plan Quality Activities



- Define quality activities and tasks throughout the SDLC.
- Plan reviews, audits, testing, and inspections.
- Define entry and exit criteria for each activity.

5 Plan Quality Resources

- Identify required skills and roles for quality activities.
- Plan training and competency development.
- Allocate tools, infrastructure, and budget for quality.

6 Define Quality Standards & Procedures

- Establish standards, guidelines, and best practices.
- Document procedures, templates, checklists, and work instructions.
- Ensure consistency and compliance.

7 Plan for Risk Management

- Identify quality-related risks.
- Analyze probability and impact.
- Plan mitigation and contingency actions.
- Monitor and review risks.

8 Plan for Measurement & Metrics

- Define quality metrics and KPIs.
- Plan data collection and analysis methods.
- Establish reporting mechanisms.
- Set up dashboards for visibility.

9 Plan for Communication & Reporting

- Define communication channels and stakeholders.
- Plan regular quality status reporting.
- Escalation paths for quality issues and risks.

10 Review & Approve the Quality Plan

- Review the Quality Plan with key stakeholders.
- Validate alignment with project objectives and requirements.
- Obtain formal approval and baseline the plan.

Key Principles

- Build quality in — not test it in.
- Prevent defects rather than detect them.
- Focus on customer satisfaction.
- Continuous improvement at every step.

Expected Outcomes

Clear Quality Direction

Reduced Defects & Rework

On-time & On-budget Delivery

Satisfied Stakeholders

Continuous Improvement

Remember

A good plan today leads to quality success tomorrow.

PART 7 OF 8

Development & Quality Plans for Small and Internal Projects

Why Small Projects Still Need Plans

- A two-week internal project still has scope, a schedule, and a definition of done the plan can be short, but skipping it entirely reintroduces the same risks at a smaller scale.
- Small projects are exactly where planning gets skipped under the assumption that 'it's too small to need this' — and exactly where that assumption is often wrong.
- A lightweight plan costs far less time to write than the rework it prevents.

Scaling Down the Development Plan

- A full-scale plan's eight elements can compress into a single page: deliverables, a short task list with dates, named owners, and the one or two risks worth tracking.
- Formal Gantt charts and detailed work-breakdown documents can give way to a simple task list, as long as dependencies are still noted.
- The goal isn't less rigor — it's the same clarity, expressed with less overhead.

Scaling Down the Quality Plan

- Quality goals can be a short list of two or three specific, testable criteria rather than a formal target matrix.
- A single informal review before delivery can replace a full review-and-audit schedule, as long as it's still scheduled and owned.
- Defect handling can be as simple as a shared task list, as long as severity and response expectations are still agreed upfront.

Full-Scale vs. Small-Project Plans

	Full-Scale Plan	Small-Project Plan
Development plan length	Multi-page document, all 8 elements	One page, compressed to essentials
Quality plan length	Multi-page document, all 8 elements	Short list of goals and one review checkpoint
Review formality	Scheduled panel reviews	Single informal peer review
Documentation	Formal, version-controlled	Lightweight, still written down

Internal Projects — Same Logic, Lighter Touch

- Internal projects follow the same scaling logic as small external ones — no external customer doesn't mean no plan.
- The 'customer' for an internal project is often another department, and they deserve the same clarity on scope and schedule an external customer would get.
- Internal quality goals should still be explicit, even if informally documented — 'we'll know it when we see it' isn't a quality goal.

PART 8 OF 8

Integration — How the Two Plans Work Together

Neither plan is complete on its own — together, they turn a contract into delivered, verified software.

Shared Inputs, Different Outputs

- Both plans start from the same source: the approved scope and estimates from contract review.
- The development plan turns that input into a sequence of construction tasks; the quality plan turns it into a sequence of verification checkpoints.
- Where the two plans disagree about scope or timeline, that disagreement should be resolved before either plan is finalized — not discovered mid-project.

Keeping the Plans Synchronized

- Every development milestone should have a corresponding entry in the quality plan's review or test schedule.
- When the development schedule changes, the quality plan's checkpoints need to move with it — a review scheduled against a milestone that no longer exists is wasted planning.
- Regular joint check-ins between the project manager and QA lead are what keep synchronization from decaying over the course of a project.

What Happens When Plans Drift Apart

- A development plan that races ahead of its quality plan produces work that accumulates unverified risk with every sprint.
- A quality plan that isn't updated alongside schedule changes ends up reviewing the wrong version of the wrong milestone.
- Drift is rarely dramatic it accumulates quietly, one small desynchronization at a time, until a release date arrives with neither plan actually describing reality.

From Plans to Delivery

Two plans, kept in step, converging on a verified release.

End of Chapter 6

A development plan without a quality plan builds fast and finds out too late. A quality plan without a development plan has nothing to check against. Together, they carry a project from contract to verified delivery.